

Etapa 3: Arquitetura por Mensagem e Nomeação de Processos

Visão Geral da Implementação

Nesta etapa, evoluímos nosso Micromundo Distribuído Interativo com foco em **comunicação assíncrona baseada em mensagens**. A arquitetura anterior, baseada em sockets diretos (Etapa 2), foi refinada para um **sistema de mensagens estruturadas** usando o **Apache Kafka** como backbone de comunicação.

O objetivo principal foi **desacoplar os componentes**, permitindo que enviem e recebam informações sem depender de respostas imediatas, aumentando a escalabilidade e a robustez do sistema. Além disso, implementamos um **mecanismo básico de nomenclatura de processos**, permitindo que os serviços do sistema se descubram dinamicamente ou via configuração estática.

Arquitetura por Mensagem e Fluxo

1. Tipos de Mensagem

No sistema, definimos dois tipos principais de mensagens:

1. **Comandos**: instruções enviadas para solicitar uma ação ou mudança de estado.
Exemplo: "CadastrarCidade" enviado pelo produtor de dados para o serviço de armazenamento.
2. **Eventos**: notificações sobre mudanças de estado ou ocorrências dentro do sistema.
Exemplo: "CidadeCadastrada" enviada após a persistência bem-sucedida da cidade na camada Bronze.

2. Estrutura das Mensagens

As mensagens seguem um formato JSON padronizado:

```
{  
  "tipo": "CadastrarCidade",  
  "payload": {
```

```
"id": 3550308,  
"cidade": "São Paulo",  
"estado": "São Paulo",  
"lat": -23.5505,  
"lon": -46.6333  
}  
}
```

- "tipo" indica o tipo de mensagem (comando ou evento).
- "payload" contém os dados específicos da operação, permitindo extensibilidade e interpretação clara pelos consumidores.

3. Fluxo de Comunicação

Fluxo principal implementado:

1. O **Producer** coleta informações de cidades via API do IBGE e envia mensagens do tipo `CadastrarCidade` para o tópico Kafka `cidades_raw`.
2. O **Spark Structured Streaming** atua como consumidor, recebendo essas mensagens de forma assíncrona, processando os dados e exibindo no console (ou armazenando na camada Bronze).
3. Após o processamento, é possível emitir eventos do tipo `CidadeCadastrada` para tópicos subsequentes, sinalizando que os dados estão prontos para camadas superiores (Silver ou Gold).

Impacto no sistema:

- A comunicação assíncrona permite que novos produtores ou consumidores sejam adicionados sem interromper o fluxo existente.
- Mensagens estruturadas garantem que todos os componentes interpretem corretamente os dados, reduzindo erros de integração.

4. Fluxo Contínuo de Dados

O sistema foi projetado para suportar **streams contínuos de dados**, permitindo atualizações em tempo real.

Exemplo: se novas cidades forem adicionadas à base IBGE ou informações geográficas forem atualizadas, novas mensagens podem ser enviadas e consumidas imediatamente, mantendo o pipeline sempre atualizado.

Mecanismo de Nomenclatura de Processos

Para que os componentes distribuídos encontrem uns aos outros, implementamos um **registro de serviços centralizado** (Service Registry) simplificado:

1. **Registro:** cada componente que inicia seu serviço se registra no servidor de nomes, informando:
 - Nome do serviço (ex.: "serviço de inventário").
 - Endereço IP e porta onde está disponível.
 - Tipo de serviço ou função.
2. **Descoberta:** componentes que precisam se comunicar consultam o registro para obter o endereço do serviço desejado antes de enviar mensagens.

Exemplo prático:

Ao iniciar, o serviço de inventário registra:

```
"serviço": "Inventário",  
"ip": "192.168.0.101",  
"porta": 9092
```

-
- Uma “formiga” (ou outro componente consumidor) consulta o registro para localizar o serviço de inventário antes de enviar um comando.

Benefícios:

- Permite adicionar ou remover serviços dinamicamente.
- Facilita manutenção e escalabilidade.
- Mantém a comunicação desacoplada e flexível.

Instruções de Construção e Execução

1. Iniciar Kafka e Zookeeper:

```
# Zookeeper
.\bin\windows\zookeeper-server-start.bat
.\config\zookeeper.properties

# Kafka broker
.\bin\windows\kafka-server-start.bat .\config\server.properties
```

2. Criar tópico cidades_raw:

```
.\bin\windows\kafka-topics.bat --create --topic cidades_raw
--bootstrap-server localhost:9092 --partitions 1
--replication-factor 1
```

3. Rodar Producer Python (envia mensagens para Kafka):

```
python producer_cidades.py
```

4. Rodar Spark Structured Streaming (consome mensagens):

```
spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12
spark_consumer.py
```

✅ Com essa implementação, temos:

- **Arquitetura baseada em mensagens**, clara e desacoplada.
- **Comunicação assíncrona**, pronta para fluxos contínuos de dados.
- **Mecanismo de nomenclatura de processos**, permitindo descoberta de serviços.
- Fluxo completo **Producer → Kafka → Spark Streaming**, pronto para evoluir para etapas Silver/Gold da arquitetura medalhão.

